Agriculture RAG and Local Model Plan

Plan and operating procedure for building a local agriculture knowledge base with Ollama or LM Studio

Objective	Create a source-backed agricultural assistant that answers exact numerical questions through structured queries and explains schemes, agronomy and data definitions through retrieval-augmented generation.
Primary hardware	DGX Spark-class system with 128 GB unified memory and approximately 2 TB storage for larger local models and fine-tuning; the MacBook Air M5 with 32 GB memory remains suitable for 7B to 14B model development and inference.
Initial scope	PMFBY, ICRISAT, APEDA, the Dharwad and Madurai workbooks, and the local agriculture source assessment.

Decision

Start with a hybrid RAG system. Keep changing agricultural statistics in a structured database, keep policies and explanations in a vector index, and use the local model to route questions and explain retrieved evidence. Fine-tuning is a later optimization, not the first ingestion step.

This procedure is designed for a local-first deployment. It can run on the Mac during development and move to the DGX Spark-class system for larger models, fine-tuning and shared access.

1. Goals and boundaries

The first release should answer agricultural questions with traceable evidence. It should distinguish facts from interpretations and should never invent a numerical value when the source is missing or ambiguous.

In scope

- Village, taluk, district and state-level agriculture records.
- NDVI, rainfall, soil moisture, acreage, production and crop-health categories.
- PMFBY scheme, insurance-unit, notification and historical-yield information where officially accessible.
- ICRISAT data catalogues, district-level data and linked datasets.
- APEDA India production and crop-category data.
- Source citations, retrieval timestamps, units and quality flags.

Out of scope for the first release

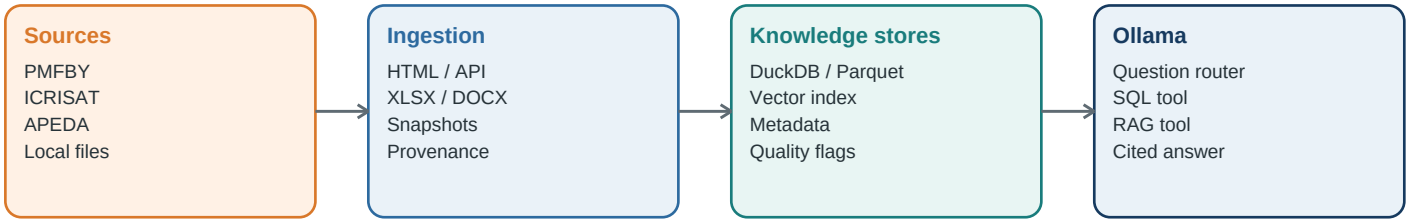
- Full pretraining of a language model from scratch.
- Treating vector similarity as the authority for exact historical numbers.
- All-India ingestion before the first state and district workflows are validated.
- Unsupervised fine-tuning directly on raw spreadsheets or scraped HTML.

Current assets

Asset	Role in the project	Observed scope
Dharwad workbook	Historical analytical records	12 sheets; 2017-2022 NDVI, rainfall, village acreage and stressed-village views
Madurai workbook	Current crop-health and weather records	29 sheets; village paddy acreage, NDVI health, rainfall and soil moisture
Source assessment DOCX	Architecture and source hierarchy	Recommends structured storage for numerical records and vector RAG for narrative knowledge

2. Target architecture

The architecture separates deterministic numerical retrieval from semantic document retrieval. This is the central design decision for accuracy.



Question routing rules

Question type	Primary tool	Expected answer behavior
Exact value or comparison	SQL / DuckDB	Return value, unit, geography, period and source
Policy or scheme explanation	Vector RAG	Retrieve authoritative passages and explain them
Trend or diagnosis	SQL plus RAG	Calculate the trend, then use definitions to explain it
Missing or conflicting evidence	Validation path	State the limitation and request a narrower question

Recommended local components

- Model runtime: Ollama or LM Studio during Mac development; a CUDA-capable runtime on the DGX for larger models.
- Structured store: DuckDB and Parquet for the proof of concept; PostgreSQL when concurrent users and permissions are required.
- Vector store: Chroma or Qdrant for a local proof of concept; pgvector when structured and vector data should share one service.
- Application layer: a small Python service that handles ingestion, routing, retrieval, citations and evaluation.

3. Source ingestion procedure

Each source must be captured as a versioned snapshot before it is transformed. Never overwrite the raw copy. Use a manifest that records URL, retrieval time, source title, checksum, file type, authority and processing status.

Source	Ingestion method	Important control
PMFBY dashboard	Capture official API responses or downloadable reports behind the JavaScript dashboard	Record session requirements, endpoint, parameters, season and retrieval date
ICRISAT data resources	Follow the linked DataVerse, GIS, forecast and district-level data resources	Store dataset metadata, licence or access notes and original download links
APEDA production	Capture the India Production catalogue and product detail responses	Preserve product category, crop name, code, year, unit and detail-page URL
Local Excel files	Parse each worksheet into raw tabular extracts	Preserve workbook name, sheet name, cell range, formulas and source notes
Local DOCX	Extract paragraphs and tables	Preserve headings, references and the source hierarchy

Raw-data directory layout

- data/raw/web/pmfby/YYYY-MM-DD/
- data/raw/web/icrisat/YYYY-MM-DD/
- data/raw/web/apeda/YYYY-MM-DD/
- data/raw/local/workbooks/
- data/raw/local/documents/
- data/manifests/

Do not embed raw HTML tables as the only representation of numbers. Convert them into canonical records and retain the source page as provenance.

4. Data normalization and quality control

The normalization stage converts different source layouts into one canonical model while retaining original labels and source context.

Field group	Minimum fields
Geography	state, district, taluk_or_tehsil, village, geography_level, source_code
Agriculture	crop, crop_standard_name, season, year, date
Observation	metric, value, unit, aggregation, estimate_type
Provenance	source_id, source_url, source_file, worksheet, cell_range, retrieved_at
Quality	revision_status, validation_status, missing_reason, confidence_note

Required validation checks

Check	Procedure	Failure action
Blank versus zero	Keep missing, zero and not-applicable as different states	Do not silently convert blanks to zero
Units	Normalize acres, hectares, millimetres and yield units	Store original unit and converted unit
Dates	Convert Excel serial dates and text dates to ISO dates	Flag ambiguous dates for review
Geography	Build crosswalks for current and historical administrative names	Prevent false trends caused by boundary changes
Duplicates	Check repeated village-crop-period keys	Retain revision information and resolve precedence
Workbook labels	Compare sheet titles with data content	Flag mismatches, such as district-name inconsistencies

Quality gates

- No record enters the production index without a source identifier.
- Every numerical answer must be traceable to source, geography, period and unit.
- Conflicting records remain visible as revisions or conflicts; they are not silently averaged.
- The original value and any transformed value are both retained.

5. Indexing and retrieval procedure

Use separate pipelines for structured facts and narrative documents.

Structured fact pipeline

- Load normalized records into DuckDB or PostgreSQL.
- Create indexes on state, district, taluk, village, crop, season, year and metric.
- Expose safe parameterized query functions rather than unrestricted SQL generation.
- Return a compact evidence table to the model with the source metadata attached.

Document RAG pipeline

- Convert source pages, policy documents, scheme guidelines and definitions into clean text.
- Chunk by heading and semantic section rather than arbitrary spreadsheet row count.
- Attach metadata for source, authority, crop, scheme, geography, language and effective date.
- Create embeddings locally and store the text chunk plus source citation.
- Use retrieval filters before similarity search when the question names a crop, state, year or scheme.

Answer format

Answer component	Required content
Direct answer	A concise sentence or calculated result
Evidence	Rows or passages used to support the answer
Context	Definition, comparison or interpretation when needed
Citation	Source name, URL or local file, period, geography and unit
Limitation	Missing data, conflicting revisions or unsupported inference

6. Local model procedure

The MacBook Air M5 with 32 GB unified memory is sufficient for the first development cycle. Use a quantized 7B or 8B model for fast iteration and test a 14B model for stronger reasoning. The DGX Spark-class device can host larger models and fine-tuning workloads.

Workload	MacBook Air M5, 32 GB	DGX Spark-class, 128 GB
7B or 8B quantized inference	Comfortable	Comfortable
14B quantized inference	Feasible; slower with long context	Comfortable
32B quantized inference	Not the first target	Practical target
LoRA fine-tuning	Start with 7B; 14B is constrained	7B-70B depending on setup
Full pretraining	Not practical	Not the initial objective

Runtime setup procedure

- Choose one primary runtime for reproducibility. Use Ollama for a simple local API or LM Studio for a desktop-first workflow.
- Download one instruct model and one embedding model. Keep model names and quantization in the project manifest.
- Set the model context length conservatively during the first evaluation.
- Start the RAG service locally and test retrieval before testing answer quality.
- Move the same ingestion and evaluation code to the DGX rather than rebuilding the project there.

Operational note

The Mac currently has LM Studio installed, while the Ollama command was not detected during the initial device check. The plan can proceed with LM Studio first or with Ollama after installation.

7. Fine-tuning procedure

Fine-tuning should be considered only after the RAG system has a measured failure pattern. It is appropriate for behavior and formatting, not for storing constantly changing agricultural facts.

Step	Procedure	Exit condition
1. Collect failures	Save questions where routing, retrieval or answer formatting failed	At least 50 labelled failures
2. Write examples	Create instruction, evidence, answer and citation examples	Examples reviewed for numeric correctness
3. Split data	Use train, validation and held-out test sets by question family	No leakage between near-duplicate questions
4. Train adapter	Run LoRA/QLoRA on the DGX; start with a small model	Validation loss and answer quality improve
5. Compare	Compare base model plus RAG against tuned model plus RAG	Held-out accuracy improves without citation loss
6. Deploy	Keep the adapter, base model, prompt and dataset versioned	Rollback path is tested

Do not fine-tune yet if

- The main problem is missing data connectors.
- The answer contains the wrong number because the database schema is incomplete.
- The model has no source citation because the retrieval response does not include provenance.
- The source itself is inconsistent or not authoritative.

8. Evaluation procedure

Evaluation must test both retrieval and answer correctness. A fluent response is not a successful response if the number, unit, year or geography is wrong.

Evaluation area	Example measure	Pass condition
Structured accuracy	Acreage, rainfall, NDVI, soil moisture	Exact or tolerance-based match to the source record
Retrieval recall	Relevant policy or definition passage	Required source appears in top-k results
Citation completeness	Source, period, geography and unit	All required metadata is present
Abstention	Unsupported question	Model says evidence is unavailable
Boundary handling	Historical versus current district	Correct geography version is identified
Regression	Previously passing questions	No degradation after ingestion or model changes

Initial golden test set

- 10 exact numerical queries from the Dharwad workbook.
- 10 exact numerical queries from the Madurai workbook.
- 10 source-definition questions about NDVI, rainfall and soil moisture.
- 10 PMFBY policy and scheme questions.
- 10 APEDA and ICRISAT source-navigation questions.
- 10 deliberately unsupported or ambiguous questions.

The first acceptance target is 90 percent exactness on the structured numerical set, with zero fabricated citations on unsupported questions.

9. Implementation schedule

Stage	Work	Deliverable
Stage 0	Freeze scope, define questions and create the source manifest	Project specification and golden-question list
Stage 1	Extract local workbooks and DOCX	Raw snapshots and normalized tables
Stage 2	Implement PMFBY, ICRISAT and APEDA connectors	Repeatable source ingestion jobs
Stage 3	Build DuckDB and vector indexes	Local knowledge base
Stage 4	Add question router and citation-aware answer prompt	Working local RAG assistant
Stage 5	Run evaluation and repair schema or retrieval issues	Evaluation report and corrections
Stage 6	Move to DGX and test larger models	Local service for larger-model inference
Stage 7	Optional LoRA fine-tuning	Versioned adapter only if justified by failures

Recommended first milestone

Complete one end-to-end workflow for Madurai: ask a village or taluk question, retrieve an exact row from DuckDB, add the relevant NDVI definition from the document index, and return a cited answer. Once that works, repeat the pattern for Dharwad and then add the external sources.

10. Operational checklist

Before ingestion	Source URL or file recorded; authority known; retrieval date recorded; raw copy preserved
Before indexing	Schema mapped; units checked; blanks and zeros separated; geography reviewed; duplicates flagged
Before answering	Question type routed; source filters applied; evidence returned with metadata
Before release	Golden set passes; citations render correctly; unsupported questions abstain; rollback tested

Final operating principle

The model explains the evidence. The database owns the number. The source manifest owns the provenance. The evaluation set owns the release decision.

11. Best-practice data onboarding SOP

Every new document or dataset should pass through the same controlled lifecycle. The goal is to make ingestion repeatable, auditable and reversible while preserving the source exactly as received.

Layer	What is stored	Rule
Intake	Source request, owner, authority, licence, intended use	Reject unknown provenance or unclear usage rights
Bronze	Immutable original file, HTML, JSON, PDF or API response	Never edit; record checksum and retrieval time
Silver	Parsed tables, cleaned text and normalized dates	Keep original fields beside transformed fields
Gold	Validated canonical records with geography, crop, period and units	Only quality-approved records enter production queries
Index	DuckDB facts, document chunks and embeddings	Version the index against the gold data release

Standard procedure

1. **Register**

   Create a source manifest entry with source ID, owner, authority, URL or file name, access date, licence note, checksum, expected geography and expected metrics.

2. **Preserve**

   Copy the source into the raw layer. Store API payloads, downloaded files and the original document. Do not clean in place.

3. **Profile**

   Measure file type, sheet or page count, row counts, columns, null rates, date ranges, units, duplicate keys and candidate geographic levels.

4. **Extract**

   Parse tables, paragraphs and API responses into a staging representation. Keep sheet name, page number, row number or API parameters as location evidence.

5. **Normalize**

   Map column names to the canonical schema, standardize crop and geography names, convert dates and units, and preserve the source value.

6. **Validate**

   Run schema, range, uniqueness, referential, unit and reconciliation checks. Produce a machine-readable error report.

7. **Publish**

   Write only approved records to the gold tables and document chunks. Rebuild the vector index from the same release.

8. **Evaluate**

   Run the golden-question set, compare against the previous release, review failures and publish a release note.

12. Format-specific cleaning rules

Different source formats fail in different ways. Use format-specific handling before applying common schema and quality checks.

Format	Clean and extract	Common failure to prevent
XLSX / XLS	Detect header rows; resolve merged cells; preserve formulas and displayed values; convert Excel dates; separate notes from data tables	Treating a dashboard layout or subtotal row as ordinary observations
CSV / TSV	Detect encoding and delimiter; validate column count; preserve leading zeros in codes; normalize null markers	Dropping village or LGD codes because they look numeric
DOCX / PDF	Extract headings, paragraphs, tables and page references; OCR only when necessary; retain page or table location	Embedding a long document without section metadata
HTML	Capture visible text, tables, links and retrieval timestamp; save the original HTML; record JavaScript/API dependencies	Assuming static HTML contains dynamic dashboard data
JSON / API	Store raw response and request parameters; validate response schema; preserve pagination and status codes	Mixing revisions or dropping API filters from provenance
Geo files	Validate CRS, geometry validity and administrative codes; keep geometry version	Joining current boundaries to historical records without a crosswalk

Canonical data contract

Field	Required behavior
source_id	Stable identifier for the source snapshot or release
geography_level	Explicit value such as village, taluk, district or state
geography_name and code	Store display name and source code; never use name alone for joins
crop_standard_name	Controlled vocabulary plus original crop label
period	Year, season and exact date where available
metric, value, unit	Typed value, normalized unit and original unit
estimate_type	Observed, administrative, remote-sensing, modelled or reported
revision_status	Original, revised, superseded or conflicting
evidence_location	Worksheet and row, page and table, URL, API parameters or cell range

13. Quality gates and release policy

A dataset is ready for the knowledge base only when it passes the gates below. Failed records remain available in quarantine for review but are excluded from production answers.

Gate	Required check	Release decision
Provenance	Source, authority, retrieval date and checksum exist	Block if missing
Schema	Required fields exist and types are valid	Quarantine invalid rows
Units	Metric units are known and conversions are documented	Block unknown units for numeric answers
Geography	Names and codes resolve through a versioned crosswalk	Flag unresolved or changed boundaries
Time	Year, season and date are consistent with source scope	Quarantine impossible or ambiguous periods
Duplicates	Duplicate keys are resolved by revision or explicitly retained as conflicts	Do not silently average
Reconciliation	Totals agree with source control totals within defined tolerance	Block material mismatches
Retrieval	Expected questions return the intended record or passage	Block if recall or ranking regresses

Recommended dataset release package

- Raw snapshot or immutable source reference.
- Normalized data export and schema version.
- Source manifest with checksum and licence note.
- Validation report listing passed, failed and quarantined records.
- Gold table or document-chunk count and index version.
- Golden-question evaluation results and known limitations.
- Release note describing additions, corrections and superseded records.

Do not publish a source as trusted merely because parsing succeeded. Parsing success is a technical status; authority, completeness and reconciliation are data-quality decisions.

References and source notes

Source	Use in this plan
PMFBY administrator statistics dashboard - https://pmfby.gov.in/adminStatistics/dashboard	Source for scheme and insurance statistics ingestion
ICRISAT data resources - https://icrisat.org/resources/data/comparative-map-homepage	Source for linked DataVerse, GIS and district-level resources
APEDA India Production - https://agriexchange.apeda.gov.in/Production/India/Index	Source for India production and crop catalogue ingestion
DHARWAD_District_Historical_Analysis_2017-2022_NDVI_RAINFALL.xlsx	Local NDVI, rainfall, acreage and stressed-village data
Madurai_Paddy_Crop_Acreages_NDVI_Health_01-15Oct-2026_Weather_NDVI_SM.xlsx	Local paddy acreage, NDVI health, rainfall and soil-moisture data
India_Agriculture_Data_Sources_for_Ollama.docx	Local source assessment and hybrid architecture recommendations
NVIDIA DGX Spark hardware guide - https://docs.nvidia.com/dgx/dgx-spark/hardware.html	Hardware capability reference for local inference and fine-tuning

Prepared for the initial agriculture knowledge-base and local-model project. Review access permissions, source terms and redistribution conditions before publishing downloaded data.